



Step 1: Import Libraries

```
In [1]: import pandas as pd
import numpy as np
import re
import nltk

from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.naive_bayes import MultinomialNB
from sklearn.svm import LinearSVC

from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay
)

import matplotlib.pyplot as plt
import seaborn as sns
```

Step 2: Download NLTK Resources

```
In [2]: nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('omw-1.4')
```

```
[nltk_data] Downloading package punkt to
[nltk_data] C:\Users\abhis\AppData\Roaming\nltk_data...
[nltk_data] Unzipping tokenizers\punkt.zip.
[nltk_data] Downloading package stopwords to
[nltk_data] C:\Users\abhis\AppData\Roaming\nltk_data...
[nltk_data] Unzipping corpora\stopwords.zip.
[nltk_data] Downloading package wordnet to
[nltk_data] C:\Users\abhis\AppData\Roaming\nltk_data...
[nltk_data] Downloading package omw-1.4 to
[nltk_data] C:\Users\abhis\AppData\Roaming\nltk_data...
```

```
Out[2]: True
```

Step 3: Load Dataset

```
In [4]: df = pd.read_csv("IMDB Dataset.csv")
```

```
df.head()
```

Out[4]:

	review	sentiment
0	One of the other reviewers has mentioned that ...	positive
1	A wonderful little production. The...	positive
2	I thought this was a wonderful way to spend ti...	positive
3	Basically there's a family where a little boy ...	negative
4	Petter Mattei's "Love in the Time of Money" is...	positive

Step 4: Basic Information

```
In [5]: df.info()  
df['sentiment'].value_counts()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 50000 entries, 0 to 49999  
Data columns (total 2 columns):  
#   Column      Non-Null Count  Dtype  
---  ---  
0   review      50000 non-null   object  
1   sentiment    50000 non-null   object  
dtypes: object(2)  
memory usage: 781.4+ KB
```

```
Out[5]: sentiment  
positive      25000  
negative      25000  
Name: count, dtype: int64
```

Step 5: Text Preprocessing Pipeline

```
In [6]: ## create stop words  
  
stop_words = set(stopwords.words('english'))  
  
negation_words = {  
    'not',  
    'no',  
    'nor',  
    'never'  
}  
  
stop_words = stop_words - negation_words
```

```
In [7]: ## Initialize Lemmatizer  
lemmatizer = WordNetLemmatizer()
```

```
In [8]: ## Text Cleaning Function
```

```
def preprocess(text):

    text = text.lower()

    text = re.sub(r'<.*?>', ' ', text)

    text = re.sub(r'[^a-zA-Z]', ' ', text)

    words = word_tokenize(text)

    words = [
        lemmatizer.lemmatize(word)
        for word in words
        if word not in stop_words
    ]

    return ' '.join(words)
```

```
In [ ]: import nltk
nltk.download('punkt_tab')

# Now apply preprocessing (this should work after downloading the resource)
df['clean_review'] = df['review'].apply(preprocess)
```

```
[nltk_data] Downloading package punkt_tab to
[nltk_data] C:\Users\abhis\AppData\Roaming\nltk_data...
[nltk_data] Unzipping tokenizers\punkt_tab.zip.
```

Step 6: Encode Target Variable

```
In [11]: df['sentiment'] = df['sentiment'].map({
    'negative':0,
    'positive':1
})
```

Step 7: Train-Test Split

```
In [12]: X_train, X_test, y_train, y_test = train_test_split(
    df['clean_review'],
    df['sentiment'],
    test_size=0.2,
    random_state=42,
    stratify=df['sentiment']
)
```

Step 8: TF-IDF Vectorization

```
In [13]: tfidf = TfidfVectorizer(
    max_features=10000,
    ngram_range=(1,2),
```

```

    min_df=2
)

X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)

```

Model 1: Multinomial Naive Bayes

In [14]: `nb = MultinomialNB()`

```

nb.fit(
    X_train_tfidf,
    y_train
)

pred_nb = nb.predict(
    X_test_tfidf
)

```

In [15]: `## Evaluation`

```

print(
    classification_report(
        y_test,
        pred_nb
    )
)

```

	precision	recall	f1-score	support
0	0.88	0.85	0.87	5000
1	0.86	0.89	0.87	5000
accuracy			0.87	10000
macro avg	0.87	0.87	0.87	10000
weighted avg	0.87	0.87	0.87	10000

In [16]: `## Confusion Matrix`

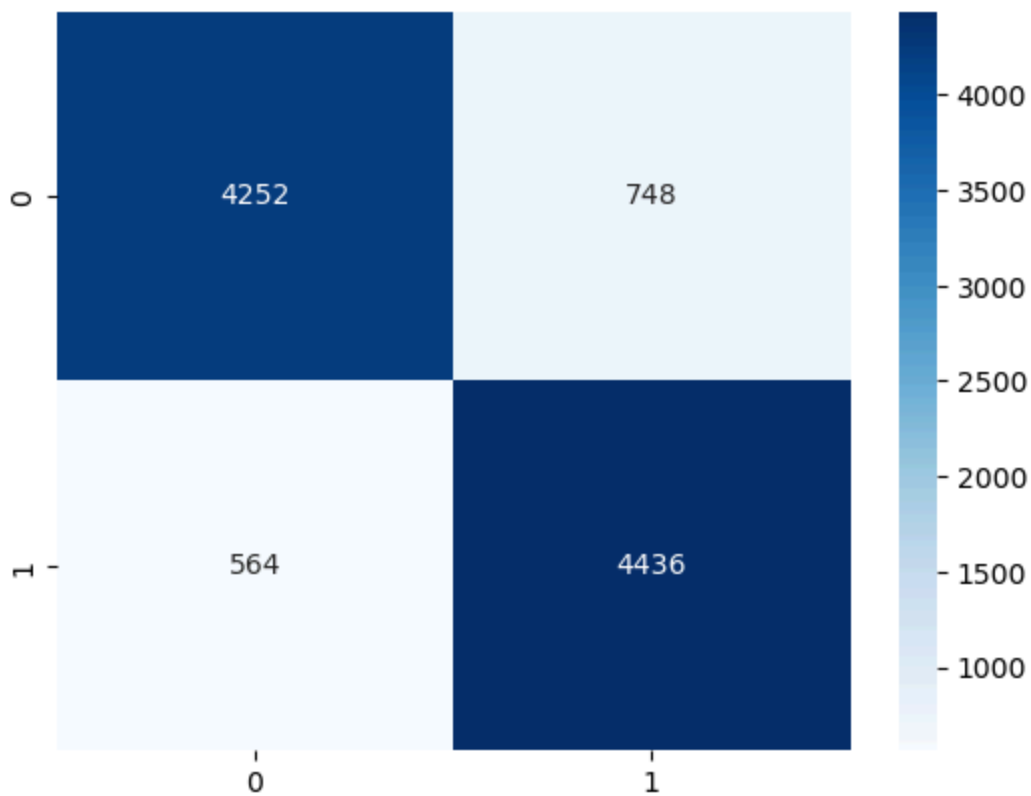
```

cm = confusion_matrix(
    y_test,
    pred_nb
)

sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues'
)

plt.show()

```



Model 2: Support Vector Machine

```
In [17]: svm = LinearSVC()

svm.fit(
    X_train_tfidf,
    y_train
)

pred_svm = svm.predict(
    X_test_tfidf
)
```

```
In [18]: ## Evaluation
print(
    classification_report(
        y_test,
        pred_svm
    )
)
```

	precision	recall	f1-score	support
0	0.90	0.89	0.89	5000
1	0.89	0.90	0.89	5000
accuracy			0.89	10000
macro avg	0.89	0.89	0.89	10000
weighted avg	0.89	0.89	0.89	10000

```
In [19]: ## Compare Models
print(
    "Naïve Bayes Accuracy:",
    accuracy_score(
        y_test,
        pred_nb
    )
)

print(
    "SVM Accuracy:",
    accuracy_score(
        y_test,
        pred_svm
    )
)
```

Naïve Bayes Accuracy: 0.8688
SVM Accuracy: 0.8936

Predict on New Reviews

```
In [20]: review = """
This movie was absolutely amazing.
The acting was fantastic.
"""

review = preprocess(review)
review = tfidf.transform([review])

prediction = svm.predict(review)

if prediction[0] == 1:
    print("Positive Review")
else:
    print("Negative Review")
```

Positive Review

Example Predictions

```
In [21]: reviews = [
    "This movie is fantastic and wonderful",
    "I hated this movie. Waste of time.",
    "The acting was not good",
    "Amazing storyline and excellent direction"
]

for r in reviews:

    clean = preprocess(r)

    vec = tfidf.transform([clean])

    pred = svm.predict(vec)[0]

    if pred == 1:
        print(r,"-> Positive")
    else:
        print(r,"-> Negative")
```

```
This movie is fantastic and wonderful -> Positive
I hated this movie. Waste of time. -> Negative
The acting was not good -> Negative
Amazing storyline and excellent direction -> Positive
```

```
In [ ]:
```